

LoxBerry Text2Speech (T2S)

Entwickler-Handbuch für das MQTT Plugin-Interface

MQTT-Plugin-Interface für LoxBerry

Aktuelle Schnittstellendokumentation für Plugin-Entwickler.

Wichtig: Text2Speech verwendet ausschließlich den in LoxBerry konfigurierten MQTT-Broker. Der T2S-Subscriber läuft als eigener systemd-Dienst und verarbeitet Requests auf tts-publish/#.

1. Übersicht

Das Text2Speech-Plugin stellt anderen Plugins ein validiertes MQTT-Interface zur Erzeugung von TTS-Dateien bereit. Andere Plugins veröffentlichen einen JSON-Request auf dem normalen LoxBerry-MQTT-Broker. Der dauerhaft laufende T2S-Subscriber validiert den Request, erzeugt bzw. lädt die MP3-Datei aus dem Cache und veröffentlicht die Antwort auf dem gewünschten Rückkanal.

- Die bestehende MQTT-Konfiguration von LoxBerry bleibt maßgeblich.

2. Laufzeitkomponenten

Komponente	Pfad / Name	Aufgabe
Subscriber	/opt/loxberry/bin/plugins/text2speech/mqtt/mqtt-subscribe.php	Validierung, TTS-Erzeugung, Cache und MQTT-Antwort
systemd-Dienst	mqtt-service-tts.service	Startet den Subscriber als Benutzer/Gruppe loxberry und startet ihn bei Abbruch neu
Broker	LoxBerry MQTT Broker	Transport der Request- und Response-Topics; wird nicht vom T2S-Plugin umkonfiguriert

3. MQTT-Topics und Rückkanal

Der Subscriber hört ausschließlich auf:

```
tts-publish/#
```

Empfohlen ist ein Request-Topic mit Plugin- und Korrelationskennung:

```
tts-publish/<client>/<corr>
```

Für die Antwort sollte jeder Request ein eindeutiges reply_to angeben:

```
tts-subscribe/<client>/<corr>
```

Ist reply_to gesetzt, verwendet T2S dieses Topic exakt. Fehlt reply_to, verwendet T2S als Fallback tts-subscribe/<client>. Fehlt auch client, lautet der Fallback tts-subscribe/text2sip.

Hinweis zu corr: Das Feld corr ist weiterhin als String erlaubt, wird vom aktuellen T2S-Subscriber aber nicht zur Antwortzuordnung ausgewertet und nicht in die Erfolgsantwort zurückgespiegelt. Für eine eindeutige Zuordnung deshalb reply_to mit einem eindeutigen Topic verwenden.

4. Request-Payload und Validierung

Der Payload muss gültiges JSON sein. Unbekannte Keys werden abgewiesen. Mindestens text oder function muss mit einem nichtleeren Wert vorhanden sein.

Feld	Typ	Pflicht	Bedeutung
text	string	text ODER function	Zu sprechender Text.
function	string	text ODER function	Addon-Funktion, siehe Abschnitt 5.
nocache	number	nein	0 = Cache verwenden, 1 = MP3 erzwingen/neu erzeugen.
logging	number	nein	1 = T2S-Logzeilen in der Erfolgsantwort mitsenden.
mp3files	number	nein	1 = Liste verfügbarer MP3-Dateien mitsenden.
client	string	nein	Kennung des anfragenden Plugins; auch Fallback für das Response-Topic.
instance	string	nein	Optionale Instanzkennung; wird in original zurückgespiegelt.
corr	string	nein	Erlaubte Korrelationskennung; aktuell nicht vom Subscriber ausgewertet.
reply_to	string	empfohlen	Exaktes MQTT-Topic für die Antwort.

Beispiel-Request

Topic: tts-publish/myplugin/7f60f8e9

```
{
  "client": "myplugin",
  "corr": "7f60f8e9",
  "reply_to": "tts-subscribe/myplugin/7f60f8e9",
  "text": "Dies ist eine Testausgabe",
  "nocache": 0,
  "logging": 1,
  "mp3files": 0
}
```

Zusätzliche Validierungsregeln:

- Ungültiges JSON wird mit status=error beantwortet.
- Nicht erlaubte Keys führen zu „Invalid keys in JSON“.
- Falsche Datentypen führen zu „Invalid types“.
- text und function dürfen nicht beide leer oder nicht vorhanden sein.
- Strings werden vor der Verarbeitung bereinigt; nicht erlaubte Steuer-/Sonderzeichen werden entfernt.
- Ein byte-identischer Payload wird innerhalb eines 25-Sekunden-Fensters als Duplikat ignoriert.

5. Funktionsaufrufe

Statt eines statischen text kann function verwendet werden. Der Subscriber ruft die jeweilige T2S-Addon-Funktion auf und übergibt den erzeugten Text anschließend an die normale TTS-Erzeugung.

function	Bedeutung
weather	Wettertext
clock	Uhrzeit
warning	Wetterwarnung

pollen	Pollenflug
abfall	Abfall-/Müllkalender

Beispiel Funktionsrequest

```
{
  "client": "myplugin",
  "reply_to": "tts-subscribe/myplugin/weather-001",
  "function": "weather",
  "logging": 1
}
```

6. Erfolgsantwort

Bei erfolgreicher Erzeugung oder erfolgreichem Cache-Treffer veröffentlicht T2S JSON mit status=done auf dem ermittelten Response-Topic. Die konkrete URL hängt von der T2S-Systemkonfiguration ab.

```
{
  "status": "done",
  "message": "MP3 file created successfully.",
  "file": "<hash>.mp3",
  "httpinterface": "<configured interface>",
  "httphostinterface": "<configured host interface>",
  "httpmp3interface": "<configured mp3 interface>",
  "cifsinterface": "<configured CIFS interface>",
  "ttspath": "/opt/loxberry/data/plugins/text2speech/tts",
  "mp3path": "/opt/loxberry/data/plugins/text2speech/mp3",
  "cifsmpp3interface": "<configured CIFS mp3 interface>",
  "timestamp": "12:34:56",
  "interfaces": {
    "httpinterface": "<configured interface>",
    "httphostinterface": "<configured host interface>",
    "httpmp3interface": "<configured mp3 interface>"
  },
  "original": {
    "client": "myplugin",
    "instance": null
  }
}
```

Wenn mp3files=1 gesetzt ist, enthält die Antwort zusätzlich mp3files. Bei logging=1 enthält sie zusätzlich logs.

7. Fehlerantworten

Validierungs- oder Laufzeitfehler werden auf dem aktuell verfügbaren Response-Topic mit status=error veröffentlicht.

```
{
  "status": "error",
  "message": "Invalid keys in JSON",
  "details": {
    "invalid_keys": ["nocach"],
    "original": { "nocach": 1 }
  },
  "logs": [],
  "timestamp": "2026-08-14T08:00:00+02:00"
}
```

Typische Fehlermeldungen sind „Invalid JSON syntax“, „Invalid keys in JSON“, „Missing mandatory field: text or function“, „Invalid types“, „Invalid function value“, „Directory not found“, „Directory not writable“ und „MP3 could not be created“.

8. Empfohlener Ablauf für aufrufende Plugins

1. Broker-Verbindungsdaten über die normale LoxBerry-MQTT-Konfiguration beziehen.
2. Eindeutige client- und corr-Werte erzeugen.
3. Eindeutiges reply_to-Topic bilden, z. B. tts-subscribe/myplugin/<corr>.

4. Zuerst genau dieses Response-Topic abonnieren.
5. Danach den JSON-Payload auf tts-publish/myplugin/<corr> veröffentlichen (QoS 0 genügt).
6. Antwort auf reply_to empfangen und status prüfen.
7. Bei status=done die zurückgelieferte Datei bzw. die konfigurierte HTTP/CIFS-Schnittstelle verwenden.
8. Subscription schließen bzw. weiterverwenden, je nach Plugin-Architektur.

Minimalbeispiel in Perl

```
use LoxBerry::IO;
use Net::MQTT::Simple;
use JSON qw(encode_json decode_json);

my $client = "myplugin";
my $corr = time();
my $req = "tts-publish/$client/$corr";
my $resp = "tts-subscribe/$client/$corr";

my $c = LoxBerry::IO::mqtt_connectiondetails();
my $mqtt = Net::MQTT::Simple->new("$c->{brokerhost}::$c->{brokerport}");
$mqtt->login($c->{brokeruser}, $c->{brokerpass}) if $c->{brokeruser} || $c->{brokerpass};

$mqtt->subscribe($resp => sub {
    my ($topic, $message) = @_;
    my $answer = decode_json($message);
    # status / file auswerten
});

$mqtt->publish($req, encode_json({
    client => $client,
    corr => "$corr",
    reply_to => $resp,
    text => "Testausgabe",
    nocache => 0
}));
```

9. Service und Fehlersuche

Der T2S-Subscriber wird durch systemd verwaltet. Die wichtigsten Prüfungen sind:

```
systemctl status mqtt-service-tts.service
journalctl -u mqtt-service-tts.service -n 50 --no-pager
systemctl status mosquitto
```

Der Dienst startet /opt/loxberry/bin/plugins/text2speech/mqtt/mqtt-subscribe.php als Benutzer und Gruppe loxberry. Bei einem Abbruch wird er automatisch nach 2 Sekunden neu gestartet.

- Wenn der Subscriber keine Verbindung zum Broker aufbauen kann, versucht er die Verbindung erneut.
- Nach einem Verbindungsverlust wird tts-publish/# nach dem Reconnect erneut abonniert.

10. Sicherheit und Integrationsregeln

- Die Sicherheit und Authentifizierung richten sich ausschließlich nach der zentralen LoxBerry-MQTT-Konfiguration.
- Aufrufende Plugins sollen für T2S keine eigenen Mosquitto-Listener oder T2S-spezifischen Broker-Konfigurationen anlegen.
- Das T2S-Plugin benötigt für das Interface keine Änderungen an /etc/mosquitto.
- Für parallele Requests immer eindeutige reply_to-Topics verwenden.
- Nur dokumentierte JSON-Keys senden; additionalProperties ist deaktiviert und unbekannte Keys werden abgewiesen.